

Hybrid RAG implementation

#installing libraries

```
!pip install langchain-google-genai
!pip install sentence-transformers langchain-huggingface
!pip install langchain-community
!pip install langchain-Chroma
!pip install pypdf
!pip install ragas
```

[Show hidden output](#)

```
!pip install langchain-pinecone
```

[Show hidden output](#)

```
!pip install langchain-experimental
```

[Show hidden output](#)

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyMuPDFLoader
from langchain_chroma import Chroma
from langchain_experimental.text_splitter import SemanticChunker
from langchain_community.retrievers import BM25Retriever
from pinecone import Pinecone
from pydantic import BaseModel, Field
from sentence_transformers import CrossEncoder
import bs4

from dotenv import load_dotenv
load_dotenv()
from google.colab import userdata
import os
```

```
!pip install pymupdf
```

[Show hidden output](#)

```
# PDF Loader
docs = PyMuPDFLoader("/content/ARBL Annual-Report-2013-14.pdf").load()

#Semantic chunking
embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-base-en-v1.5")
chunks = SemanticChunker(embeddings).split_documents(docs)
```

[Show hidden output](#)

```
from pinecone import Pinecone, PodSpec
from google.colab import userdata
import os
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key=userdata.get("PINE"))
index_name = "fin-rag"

# Define the correct dimension for the index based on the embedding model
expected_dimension = 768 # BAAI/bge-base-en-v1.5 has 768 dimensions

# Check if index exists, and create if it doesn't
if index_name not in pc.list_indexes():
    print(f"Creating new Pinecone index: {index_name} with dimension {expected_dimension}")
    # IMPORTANT: You need to replace 'your-pinecone-environment' with the exact environment name from your Pinecone dashboard.
    # For AWS free-tier, it's often 'us-east-1-aws'.
    pc.create_index(
        index_name,
```

```

        dimension=expected_dimension,
        metric='cosine',
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )
else:
    print(f"Pinecone index '{index_name}' already exists.")

index = pc.Index(index_name)

# Store embeddings in Pinecone
for i, doc in enumerate(chunks):
    index.upsert(
        vectors=[{
            "id": str(i),
            "values": embeddings.embed_query(doc.page_content),
            "metadata": {
                "text": doc.page_content,
                "page": doc.metadata.get("page", 0)
            }
        }]
    )

```

Creating new Pinecone index: fin-rag with dimension 768

!pip install rank_bm25

[Show hidden output](#)

```

#Hybrid search : vector + BM25

bm25 = BM25Retriever.from_documents(chunks)
bm25.k = 5

def hybrid_search(query,k=5):
    vector = embeddings.embed_query(query)

    pinecone_result = index.query(
        vector = vector,
        top_k = k,
        include_metadata = True
    )

    bm25_result= bm25.invoke(query)

    return [
        r["metadata"]["text"]
        for r in pinecone_result["matches"]
    ] + [
        r.page_content
        for r in bm25_result
    ]

```

```

#Re-ranking

reranker = CrossEncoder("BAAI/bge-reranker-base")

def rerank(query,docs,k=5):
    scores = reranker.predict([(query,d) for d in docs])
    return [d for score, d in sorted(zip(scores,docs),reverse=True)[:k]]

```

Loading weights: 100%

201/201 [00:00<00:00, 2730.18it/s]

#Structured output

```

class Answer(BaseModel):
    answer:str
    financial_year:str
    source_pages:list[int]

llm = ChatGoogleGenerativeAI(model="gemini-3.6-flash", temperature=0,api_key=userdata.get("GEMINI")).with_structured_output(Ans

```

#Refine chain

```
def refine_chain(query, docs):
    answer = llm.invoke(
        f""" answer using the provide financial context.
        "Question": {query}
        "context": {docs[0]}
        """
    )

    for doc in docs[1:]:
        answer = llm.invoke(f""" Refine the existing answer using the additional context)
        Question:{query}
        Existing answer: {answer}
        Additional context:{docs}""")
    return answer
```

```
# Complete RAG

def financial_rag(query):
    docs = hybrid_search(query)
    docs = rerank(query, docs)

    return refine_chain(query, docs)

#Example

result = financial_rag("What was the net profit FY2013-14?")
print(result)
```

[Show hidden output](#)

```
print(result.answer)
```

For FY2013-14, the Net Profit (Profit After Tax / Profit for the year) was ₹3,674.36 million (reported as ₹3,674 million in the

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.